

L2 - 13.1 x86 Architecture Overview

Task 1 - Introduction

Malware often works by abusing the way systems are designed. Therefore, to understand how malware works, it is essential that we know the architecture of the systems they are running in.

Task 2 - CPU architecture overview

The CPU architecture that is most widely used is derived from the Von Neumann architecture. The CPU has three components: the Arithmetic Logic Unit (ALU), the Control Unit, and the Registers. The CPU interacts with memory and I/O devices outside the CPU.

Control Unit: Fetches and manages instruction execution. The address to the next instruction to execute is stored in a register called the Instruction Pointer or IP. In 32-bit systems, this register is called EIP, and in 64-bit systems, it is called RIP.

Arithmetic Logic Unit: Executes instructions and performs calculations. The results of the executed instruction are then stored in either the Registers or the Memory.

Registers: The Registers are the CPU's storage.

Memory: Holds program code and data during execution.

I/O devices: Enable communication with external hardware.

In short, when a program has to be executed, it is loaded into the memory. From there, the Control Unit fetches one instruction at a time using the Instruction Pointer Register, and the Arithmetic Logic Unit executes it. The results are stored in either the Registers or the Memory.

Answer the questions below

In which part of the Von Neumann architecture are the code and data required for a program to run stored?

What part of the CPU stores small amounts of data?

Registers

In which unit are arithmetic operations performed?

Arithmetic Logic Unit

Task 3 - Registers overview

Registers are the CPU's storage medium. The registers are divided into the following different types:

- Instruction Pointer
- General Purpose Registers
- Status Flag Registers
- Segment Registers

The Instruction Pointer:

The Instruction Pointer(IP) is a register that contains the address of the next instruction to be executed by the CPU. It is also called the Program Counter. It was originally a 16-bit register in the Intel 8086 processor and was abbreviated as IP. In 32-bit processors, the Instruction Pointer became a 32-bit register called the EIP or the Extended Instruction Pointer. In 64-bit systems, this register became a 64-bit register called RIP (the R here stands for register).

General-Purpose Registers: Perform general CPU operations. They contain the following registers.

EAX or RAX: (Accumulator): Stores arithmetic operation results.

EBX or RBX: (Base Register): Holds base addresses for memory access.

ECX or RCX: (Counter Register): Used for loops and counting.

EDX or RDX:(Data Register): Handles multiplication and division operations.

ESP or RSP: (Stack Pointer): Points to the top of the stack.

EBP or RBP:(Base Pointer): References stack parameters and local data.

ESI or RSI: (Source Index): Points to source data for string operations.

EDI or RDI:(Destination Index): Points to destination data for string operations.

R8-R15: Additional 64-bit general-purpose registers.

Answer the questions below

| Which register holds the address to the next instruction that is to be executed?

Instruction Pointer

| Which register in a 32-bit system is also called the Counter Register?

ECX

| Which registers from the ones discussed above are not present in a 32-bit system?

R8-R15

Task 4 - Registers - Continued

Status Flag Registers: Store the CPU execution status that can be either 1 or 0.

1. Zero Flag: Denoted by ZF, the Zero Flag indicates when the result of the last executed instruction was zero. For example, if an instruction is executed that subtracts a RAX from itself, the result will be 0. In this situation, the ZF will be set to 1.

2. Carry Flag: Denoted by CF, the Carry Flag indicates when the last executed instruction resulted in a number too big or too small for the destination. For example, if we add 0xFFFFFFFF and 0x00000001 and store the result in a 32-bit register, the result will be too big for the register. In this case, CF will be set to 1.

3. Sign Flag: The Sign Flag or SF indicates if the result of an operation is negative. If these conditions are met, the SF is set to 1; otherwise, it is set to 0.

4. Trap Flag: The Trap Flag or TF indicates if the processor is in debugging mode. When the TF is set, the CPU will execute one instruction at a time for debugging purposes.

Segment Registers: Divide memory into logical segments.

1. Code Segment: The Code Segment (CS) register points to the Code section in the memory.

2. Data Segment: The Data Segment (DS) register points to the program's data section in the memory.

3. Stack Segment: The Stack Segment (SS) register points to the program's Stack in the memory.

4.5.6. Extra Segments (ES, FS, and GS): These extra segment registers point to different data sections. These and the DS register divide the program's memory into four distinct data sections.

Answer the questions below

| Which flag is used by the program to identify if it is being run in a debugger?

Trap Flag

| Which flag will be set when the most significant bit in an operation is set to 1?

Sign Flag

| Which Segment register contains the pointer to the code section in memory?

Code Segment

Task 5 - Memory overview

Each program runs in its own **virtual memory space**, isolated from the rest of the system. This memory is divided into four main sections: **Code**, **Data**, **Heap**, and **Stack**, each serving a specific purpose. Understanding this layout is essential for malware analysis and reverse engineering.

Code: Contains executable instructions from the Portable Executable (PE) file; the CPU runs code stored in this memory section.

The Code Section, as the name implies, contains the program's code. Specifically, this section refers to the text section in a Portable Executable file, which includes instructions executed by the CPU. This section of the Memory has execute permissions, meaning that the CPU can execute the data in this section of the program memory.

Data: Stores fixed, initialized data such as global variables that do not change during program execution.

Heap: Dynamic memory area where variables are created, stored, and removed during runtime.

Stack: Stores local variables, function arguments, and return addresses; often targeted by malware to manipulate program control flow.

Answer the questions below

When a program is loaded into Memory, does it have a full view of the system memory?

N

Which section of the Memory contains the code?

Code

Which Memory section contains information related to the program's control flow?

Stack

Task 6 - Stack Layout

The stack stores function arguments, local variables, and control flow information; malware can exploit it to hijack execution flow.

The stack is a Last In First Out (LIFO) memory. This means that the last element pushed onto the stack is the first one to be popped out. For example, if we push A, B, and C onto the stack, when we pop out these elements, the first to pop out will be C, B, and then A. The CPU uses two registers to keep track of the stack. One is the Stack Pointer (the ESP or RSP), and the other is the Base Pointer (the EBP or RBP).

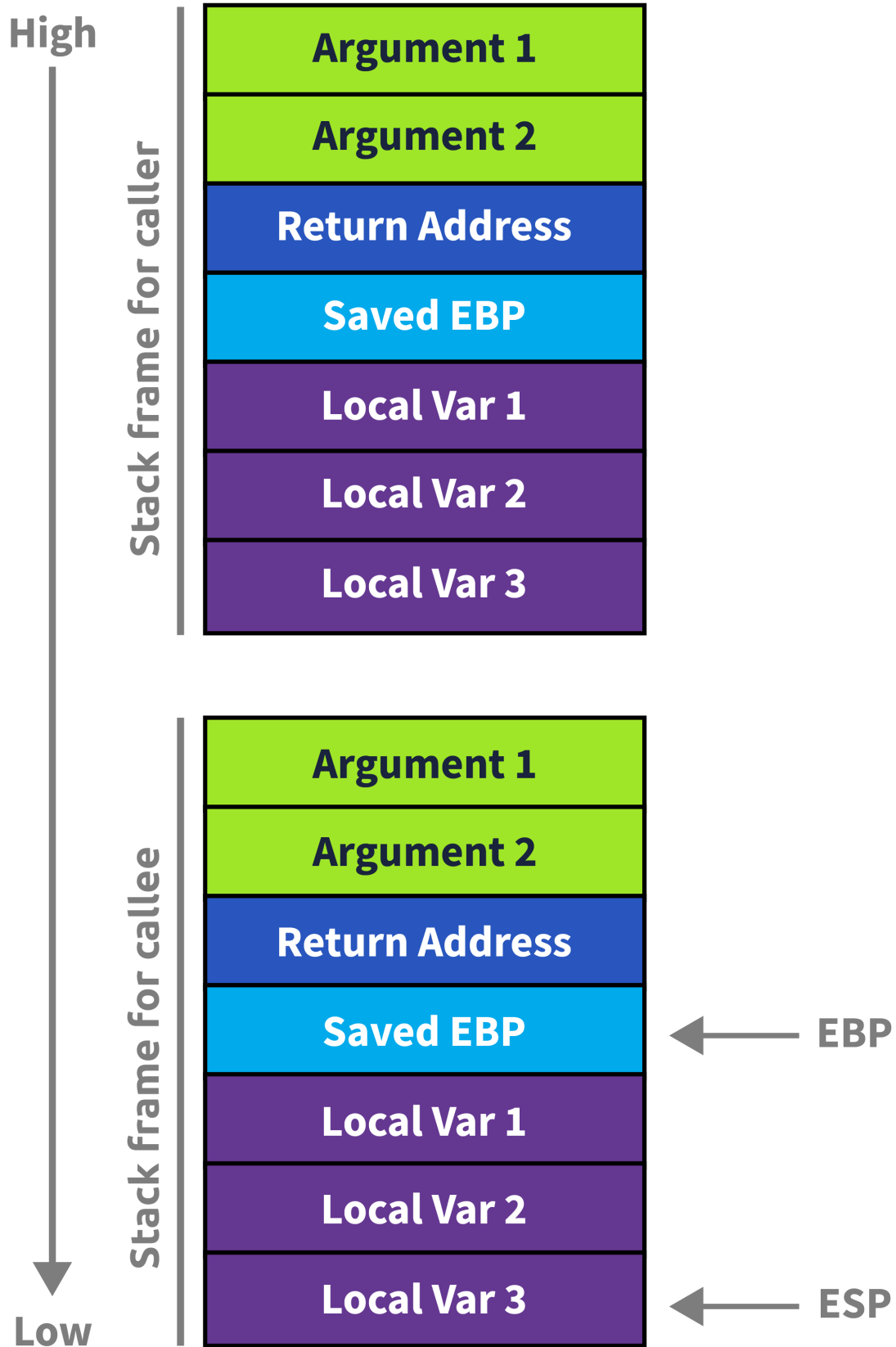
The Stack Pointer: The Stack Pointer points to the top of the stack. When any new element is pushed on the stack, the location of the Stack Pointer changes to consider the new element that was pushed on the stack. Similarly, when an element is popped off the stack, the stack pointer adjusts itself to reflect that change.

Stack (LIFO): The stack works as Last In First Out (LIFO), where the last pushed item is the first removed; it is managed by Stack Pointer (ESP/RSP) and Base

Pointer (EBP/RBP).

Stack Pointer (ESP/RSP): Points to the top of the stack and changes whenever data is pushed or popped.

The Base Pointer (EBP/RBP): Provides a fixed reference point for accessing a function's local variables and arguments.



Old Base Pointer and Return Address: Stores the previous function's stack reference and the return location; attackers can overwrite the return address using stack buffer overflow attacks to hijack control flow.

Arguments: Function parameters are stored on the stack before the function begins execution.

Function Prologue and Epilogue:

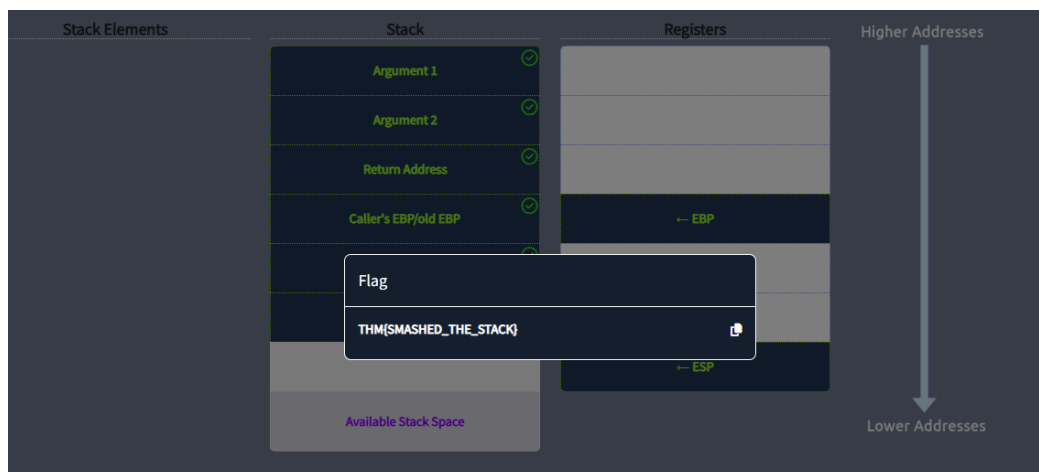
Function Prologue: The setup phase of a function where arguments, return address, and old base pointer are saved, and the stack frame is created.

Function Epilogue: The cleanup phase where the old base pointer and return address are restored, returning execution to the calling function.

Answer the questions below

Follow the instructions in the attached static site and find the flag. What is the flag?

THM{SMASHED_THE_STACK}



Task 7 - Conclusion

That concludes this room on the primer of x86-64 system architecture.

- The Von Neumann CPU architecture
- Different components of a CPU
- Different types of CPU registers

- Memory and its different sections
- Stack layout of a program in memory

Register	64-bit Name	32-bit Name	Purpose
Instruction Pointer	RIP (Register Instruction Pointer)	EIP (Extended Instruction Pointer)	Stores the address of the next instruction the CPU will execute.
Stack Pointer	RSP (Register Stack Pointer)	ESP (Extended Stack Pointer)	Points to the top of the current stack . Changes when data is pushed or popped.
Base Pointer	RBP (Register Base Pointer)	EBP (Extended Base Pointer)	Points to the base of the current stack frame and helps access local variables and function arguments.